

// Jivops

MLOps en Producción

Guía completa — 30 lecciones

Fundamentos · Datos y experimentos · Pipelines · Registro y despliegue ·
Monitoreo · Producción avanzada

Contenido

Módulo 1: Fundamentos de MLOps

- Lección 01 — Qué es MLOps y por qué existe
- Lección 02 — Ciclo de vida de un modelo de ML en producción
- Lección 03 — Niveles de madurez MLOps
- Lección 04 — Herramientas del ecosistema MLOps
- Lección 05 — Reproducibilidad: código, datos y entorno

Módulo 2: Datos y experimentos

- Lección 06 — Versionado de datos con DVC
- Lección 07 — Feature stores
- Lección 08 — Seguimiento de experimentos con MLflow Tracking
- Lección 09 — Gestión de hiperparámetros y búsqueda
- Lección 10 — Validación de datos y esquemas

Módulo 3: Pipelines y entrenamiento

- Lección 11 — Pipelines de ML como código
- Lección 12 — Kubeflow Pipelines
- Lección 13 — Orquestación con Airflow para ML
- Lección 14 — Entrenamiento distribuido y GPUs en Kubernetes
- Lección 15 — CI/CD para ML y entrenamiento continuo (CT)

Módulo 4: Registro y despliegue

- Lección 16 — Model Registry
- Lección 17 — Empaquetado de modelos: contenedores y ONNX
- Lección 18 — Serving: batch vs online
- Lección 19 — KServe y serving en Kubernetes
- Lección 20 — Estrategias de despliegue: canary, shadow y A/B

Módulo 5: Monitoreo y reentrenamiento

- Lección 21 — Monitoreo: métricas de sistema vs métricas de modelo
- Lección 22 — Data drift
- Lección 23 — Concept drift y degradación del modelo
- Lección 24 — Reentrenamiento automático y triggers
- Lección 25 — Observabilidad y logging de predicciones

Módulo 6: Producción avanzada

Lección 26 — Gobernanza, linaje y auditoría

Lección 27 — Seguridad en ML: datos sensibles y ataques

Lección 28 — Costos y optimización de inferencia

Lección 29 — LLMOps: particularidades de los LLMs en producción

Lección 30 — Proyecto final: plataforma MLOps completa lista para producción

Módulo 1: Fundamentos de MLOps

Lección 01 — Qué es MLOps y por qué existe

Lleva los modelos a producción de forma repetible, monitoreada y mantenible, aplicando prácticas de DevOps al ciclo de vida del machine learning.

En producción cambian los datos, el entorno y la escala, y sin reproducibilidad ni monitoreo nadie detecta la degradación.

Lección 02 — Ciclo de vida de un modelo de ML en producción

El comportamiento depende de datos además de código, por lo que el modelo puede degradarse aunque el código no cambie.

El mismo código con datos distintos da modelos distintos; sin versionar los tres no puedes reproducir ni auditar un resultado.

Lección 03 — Niveles de madurez MLOps

Entrenamiento y despliegue manuales, sin pipelines automatizados ni monitoreo sistemático.

Automatizar demasiado pronto añade complejidad antes de que el equipo y el caso de uso la justifiquen.

Lección 04 — Herramientas del ecosistema MLOps

Para registrar experimentos, parámetros, métricas y modelos, gestionando su ciclo de vida.

Cada etapa (datos, experimentos, pipelines, serving, monitoreo) tiene herramientas especializadas que se integran entre sí.

Lección 05 — Reproducibilidad: código, datos y entorno

Código, datos y entorno (dependencias y configuración).

El contenedor fija dependencias y versiones, así el entrenamiento se comporta igual en cualquier máquina.

Módulo 2: Datos y experimentos

Lección 06 — Versionado de datos con DVC

Versiona datasets grandes guardando metadatos ligeros en Git y los datos reales en un almacenamiento remoto.

El repositorio se vuelve enorme y lento de clonar; DVC guarda solo punteros ligeros en Git.

Lección 07 — Feature stores

Centraliza el cálculo y almacenamiento de features para reutilizarlas de forma consistente entre entrenamiento e inferencia.

Si la lógica difiere, el modelo recibe en producción datos distintos a los de entrenamiento y degrada sin avisar.

Lección 08 — Seguimiento de experimentos con MLflow Tracking

Parámetros, métricas, artefactos y la versión del código de cada ejecución.

Con un historial comparable sabes qué combinaciones ya probaste y puedes reproducir la mejor.

Lección 09 — Gestión de hiperparámetros y búsqueda

Un valor de configuración definido antes del entrenamiento que controla cómo aprende el modelo.

No todos los hiperparámetros influyen igual; muestrear al azar explora más valores relevantes con menos ejecuciones.

Lección 10 — Validación de datos y esquemas

Para detectar valores fuera de rango, columnas faltantes o cambios de esquema que degradarían el modelo.

Un dato malo detectado al inicio es fácil de localizar; si entra al entrenamiento produce un modelo defectuoso difícil de diagnosticar.

Módulo 3: Pipelines y entrenamiento

Lección 11 — Pipelines de ML como código

Es versionable, repetible y se puede ejecutar de forma automática y consistente.

Pasos modulares se reutilizan, se depuran por separado y permiten cachear resultados ya calculados.

Lección 12 — Kubeflow Pipelines

Una herramienta para definir y ejecutar pipelines de ML como workflows de contenedores sobre Kubernetes.

Cada paso lleva sus propias dependencias, se ejecuta igual en cualquier cluster y no interfiere con los demás.

Lección 13 — Orquestación con Airflow para ML

Para programar y orquestar tareas como ingesta de datos, entrenamiento y evaluación mediante DAGs.

El DAG obliga a que una tarea espere a que termine la anterior, evitando entrenar con datos incompletos.

Lección 14 — Entrenamiento distribuido y GPUs en Kubernetes

Declarando el recurso `nvidia.com/gpu` en los límites del contenedor, con el device plugin instalado en los nodos.

Son costosas; una GPU ociosa desperdicia dinero, por eso se planifica su uso y se comparte cuando es posible.

Lección 15 — CI/CD para ML y entrenamiento continuo (CT)

Validación de datos y de modelos, y en muchos casos entrenamiento continuo además de pruebas de código.

Hay que comparar contra el modelo actual con un umbral de calidad para no desplegar uno peor.

Módulo 4: Registro y despliegue

Lección 16 — Model Registry

Almacenar versiones de modelos con su metadata y estado (staging, producción), facilitando la promoción controlada.

Es la fuente única de verdad con versiones y estados claros, así se sabe exactamente qué modelo corre en producción.

Lección 17 — Empaquetado de modelos: contenedores y ONNX

Representar modelos de distintos frameworks en un formato abierto portable entre entornos de inferencia.

La imagen contiene modelo y dependencias, es un artefacto único que se ejecuta igual en cualquier cluster.

Lección 18 — Serving: batch vs online

Cuando las predicciones no requieren respuesta inmediata y pueden calcularse periódicamente sobre grandes volúmenes.

Hay un usuario esperando; una respuesta lenta o una caída se nota de inmediato, a diferencia de un batch que puede reintentarse.

Lección 19 — KServe y serving en Kubernetes

Un recurso InferenceService que gestiona despliegue, autoescalado y endpoints estandarizados para modelos.

Los recursos se ajustan a la demanda real y se liberan cuando no hay tráfico, evitando pagar capacidad ociosa.

Lección 20 — Estrategias de despliegue: canary, shadow y A/B

El nuevo modelo recibe copia del tráfico real y genera predicciones que se comparan, sin afectar a los usuarios.

Si falla, solo afecta a una fracción pequeña y se puede revertir rápido, tras validar con datos reales.

Módulo 5: Monitoreo y reentrenamiento

Lección 21 — Monitoreo: métricas de sistema vs métricas de modelo

Porque un modelo puede estar disponible y rápido pero dando predicciones cada vez peores.

El resultado real de una predicción a veces se conoce semanas después, por eso se usan métricas proxy mientras tanto.

Lección 22 — Data drift

Un cambio en la distribución de los datos de entrada respecto a los usados en el entrenamiento.

El modelo aprendió patrones de una distribución; si los datos reales cambian, esos supuestos dejan de ser válidos.

Lección 23 — Concept drift y degradación del modelo

En el concept drift cambia la relación entre las entradas y la variable a predecir, no solo la distribución de las entradas.

El modelo aprendió patrones del pasado; si el comportamiento cambia, la relación aprendida deja de ser válida y hay que reentrenar.

Lección 24 — Reentrenamiento automático y triggers

Detección de drift, caída de una métrica de calidad, llegada de nuevos datos o un calendario.

Un reentrenamiento con datos defectuosos podría producir un modelo peor y desplegarlo solo; hay que validar antes de promoverlo.

Lección 25 — Observabilidad y logging de predicciones

Permite auditar, depurar errores, detectar drift y reentrenar con datos reales.

Permite saber qué versión generó cada predicción y correlacionar el problema con un despliegue concreto.

Módulo 6: Producción avanzada

Lección 26 — Gobernanza, linaje y auditoría

El rastro que conecta un modelo con los datos, código y parámetros con los que se produjo.

Permite demostrar de dónde viene cada decisión del modelo ante auditorías y reguladores.

Lección 27 — Seguridad en ML: datos sensibles y ataques

Entradas manipuladas deliberadamente para provocar predicciones erróneas.

Con autenticación y límites de peticiones se dificulta extraer el modelo o abusar de él mediante consultas masivas.

Lección 28 — Costos y optimización de inferencia

Cuantización, que representa los pesos con menor precisión numérica.

Permite comparar el beneficio de mayor precisión contra su costo y elegir el modelo suficiente, no el más grande.

Lección 29 — LLMOps: particularidades de los LLMs en producción

Gestión de prompts, evaluación de respuestas, guardrails, costos por token y uso de RAG.

Un cambio de prompt cambia el comportamiento del sistema; versionarlo permite comparar y volver atrás.

Lección 30 — Proyecto final: plataforma MLOps completa lista para producción

Datos y modelos versionados + pipelines automatizados + registry + despliegue gradual + monitoreo de drift + gobernanza.

Menciona al menos dos pilares: versionado, pipelines automatizados, registry, despliegue gradual o monitoreo de drift.